CASE-STUDY PROJECT REPORT (ESE)

# GlassQuill

*Explainable Detection of AI-Generated Text*

**Applications and Use Cases of Machine Learning (TE7483)**

AIML Department, Symbiosis Institute of Technology, Pune

Batch 2024–28 · Semester V · October 2026

SUBMITTED BY


| Name | PRN |
|---|---|
| Ranjan Verma | 24070126149 |
| Rahul Narkhede | 24070126146 |
| Priyansh Johri | 24070126142 |
| Priyansh Pandey | 24070126143 |


SUBMITTED TO

Dr. Aniket Shahade, Dr. Shrikishna Kolhar, Dr. Sumanto Dutta

Code, models and results: `github.com/ThatOneRahul/glassquill`

# Abstract

Large language models now write text that people cannot reliably tell apart from human writing. Colleges, newsrooms and hiring teams have turned to AI-text detectors, but most of these tools give back a bare score. A student who is wrongly flagged has no way to question it, and non-native English writers get flagged far more often than others. GlassQuill classifies an English text of 50 to 400 words as human-written or AI-generated and explains every decision it makes.

We train four models that go from fully transparent to a black box: a zero-shot GPT-2 perplexity threshold, TF-IDF with logistic regression, XGBoost on 18 style and perplexity features that are easy to interpret, and a fine-tuned DistilBERT. The data combines three public corpora (HC3, HAP-E and M4GT) and covers ChatGPT from 2022 as well as recent models such as GPT-4o, GPT-4, Llama-3, Mixtral and Gemma. In total there are 31,984 texts. Each of the 17 domains has the same number of human and AI texts, and the data is split by question or document so that no topic leaks from training into testing. A first version scored above 99%, but only because of formatting shortcuts in the data. Removing them was the most important step in the project.

The deployed verdict is the mean of the two explainable models. On 4,829 unseen test texts it reaches 97.4% accuracy, 0.974 F1 and a 2.0% false positive rate. That is ahead of DistilBERT (96.1% accuracy, 7.1% false positive rate), and every verdict comes with SHAP feature reasons and word-level evidence. On MAGE, a benchmark of 27 generators never seen in training, every model falls to 61–66% accuracy. Paraphrasing lowers recall from 98% to 93%. We report both as limits. The project is built as a reproducible eight-stage pipeline and a tested web application.

**Keywords:** AI-generated text detection, explainable machine learning, perplexity, stylometry, XGBoost, SHAP, NLP text classification.

# Contents


Abstract 1

Contents 2

1. Introduction and Problem Statement 4

1.1 Problem statement 4

1.2 Real-world relevance 4

1.3 Objective 4

1.4 Research questions and hypotheses 5

1.5 Scope 5

1.6 Application domain and stakeholders 5

1.7 Refinement since Phase I 6

2. Background and Related Work 7

2.1 Key concepts 7

2.2 Related work 7

3. Dataset 9

3.1 Sources 9

3.2 Attributes, target variable and data types 9

3.3 Size, balance and exploratory analysis 10

3.4 Missing values and duplicates 10

3.5 Preprocessing and shortcut removal 11

3.6 Feature engineering 11

3.7 Feature selection 12

3.8 Validation strategy 12

4. Methodology and ML Pipeline 14

4.1 Pipeline overview 14

4.2 Model selection and justification 14

4.3 Training strategy and hyper-parameters 15

4.4 Evaluation method 15

4.5 Explanation method 15

5. Implementation 16

5.1 Tools and environment 16

5.2 Project structure 16

5.3 Workflow and reproducibility 16

5.4 The web application 17

5.5 Testing and security 18

**6. Results and Comparative Analysis** 19
6.1 In-domain test set 19
6.2 Results per corpus 20
6.3 Out-of-domain: MAGE 21
6.4 Paraphrase attack 21
6.5 Comparative analysis 21
6.6 Explanations 22
6.7 Answers to the research questions 23
**7. Limitations and Ethical Considerations** 24
**8. Conclusion and Future Scope** 25
8.1 Conclusion 25
8.2 Future scope 25
**References** 26
**Appendix A. Rubric Coverage** 27
**Appendix B. Class Balance per Domain** 28

# 1. Introduction and Problem Statement

## 1.1 Problem statement

Large language models (LLMs) such as ChatGPT, Gemini and Claude write fluent text in seconds. Places that grade or publish writing now use automatic AI-text detectors, yet almost all of them give a single probability with no explanation. When such a detector is wrong, a real person pays for it. A student accused of cheating or a writer whose work is rejected cannot see the reasoning or challenge it. Liang et al. (2023) found that popular detectors flagged more than half of the essays written by non-native English speakers as AI-generated [6].

> **Problem:** build a detector that labels English text as human-written (0) or AI-generated (1), keeps the false positive rate (humans wrongly flagged as AI) low and explains each decision in a way a non-expert can check.

## 1.2 Real-world relevance

- **Education:** universities use detectors in academic-integrity cases. With an explanation, a score becomes evidence that people can talk through.
- **Publishing and moderation:** platforms screen large numbers of submissions and need a record of why each decision was made.
- **Hiring:** more and more cover letters and written tests are written with LLM help, and employers need a fair second opinion.
- **Research:** most new detectors are black boxes, so a clear and well-documented baseline is useful for comparison.

## 1.3 Objective

Build and test an explainable binary classifier for English text of 50 to 400 words that (a) is highly accurate with a low false positive rate, (b) explains every verdict with feature-level and word-level evidence and (c) is checked for robustness on unseen generators and on paraphrased AI text. The result is delivered as a reproducible pipeline and a working web application.

## 1.4 Research questions and hypotheses

| ID | Research question |
|---|---|
| RQ1 | Which simple signals (perplexity, sentence rhythm, vocabulary, punctuation) separate human and AI text, and how strongly? |
| RQ2 | How close can an explainable model (XGBoost with SHAP) get to a black box (fine-tuned DistilBERT)? |
| RQ3 | How well do the detectors work on unseen LLMs and on paraphrased AI text? |
| RQ4 | Do two explanation methods (SHAP and LIME) agree on why a text was flagged? |

**Table 1:** Research questions

| Hypothesis | How it is tested |
|---|---|
| H1: AI text has lower perplexity | class means and SHAP direction of log_ppl |
| H2: AI text has lower burstiness (more even sentences) | class means and SHAP direction of burstiness and nll_std |
| H3: DistilBERT wins in-domain but drops most out-of-domain | F1 and FPR on the test set vs MAGE |
| H4: paraphrasing lowers recall for every model | recall on AI texts before vs after T5 paraphrasing |

**Table 2:** Hypotheses

## 1.5 Scope

- **In scope:** English text of 50 to 400 words, two classes (human and AI) and many genres: question answering, essays, news, fiction, academic writing, blogs, peer reviews, Reddit, WikiHow and Wikipedia.
- **Out of scope:** other languages and source code. We also do not handle documents that mix human and AI writing, and we do not try to name the LLM that wrote a text.

## 1.6 Application domain and stakeholders

The application domain is education and content moderation. The ML task is binary NLP text classification.

| Stakeholder | What they need from GlassQuill |
|---|---|
| Teachers and universities | a second opinion that comes with reasons, never a final verdict |
| Students and writers | a fair way to see and question why their text was flagged |
| Content platforms and newsrooms | screening of large volumes of text with a clear audit trail |
| Researchers | a transparent and reproducible baseline to compare new detectors against |

**Table 3:** Stakeholders and their needs

## 1.7 Refinement since Phase I

The Phase I proposal planned a study on HC3 alone, with XGBoost as the main model and a Gradio demo built in Google Colab. Building and testing that plan showed two serious problems, so we changed the project as shown below.

| Area | Phase I plan | Final project | Why it changed |
|---|---|---|---|
| Data | HC3 only (ChatGPT, Dec 2022), about 12,000 texts | HC3 + HAP-E + M4GT: 31,984 texts, 17 domains, 2024 models | the HC3-only model labelled a modern ChatGPT paragraph 100% human |
| Preprocessing | basic cleaning | systematic shortcut removal (Table 8) | the first full run scored above 99% by spotting formatting, not style |
| Verdict | XGBoost alone | mean of XGBoost and TF-IDF + LR, chosen on validation | false positive rate fell from 6.6% to 2.0% |
| Model behaviour | unconstrained trees | monotone constraints where theory and data agree | prevents rules like 'very even sentences mean human' |
| Implementation | Colab notebook and Gradio demo | 8-stage reproducible script, FastAPI + React web app, 28 automated tests | reproducibility, security and a usable interface |

**Table 4:** Refinement from the Phase I plan to the final project

# 2. Background and Related Work

## 2.1 Key concepts

**Perplexity**

> The exponential of the mean negative log-probability of each token under a language model. LLMs pick likely tokens, so another language model finds their text easy to predict. This means AI text has lower perplexity. We use GPT-2 small [15] as the reader model.

**Burstiness**

> The standard deviation of sentence length divided by its mean. Human writers mix short and long sentences while LLM output is more even.

**Stylometry**

> Measurable features of writing style such as vocabulary richness, word length, punctuation and use of function words.

**TF-IDF with logistic regression**

> Each text becomes a weighted bag of words and word pairs. The linear model gives every n-gram an exact weight towards one class, so it is easy to interpret by design.

**XGBoost**

> Gradient-boosted decision trees [13]. It is strong on tabular features and supports exact TreeSHAP explanations.

**DistilBERT**

> A compact transformer [14] fine-tuned end to end on the raw text. It is accurate but you cannot see inside it.

**SHAP**

> Shapley values from cooperative game theory [10, 11]. The base value plus the sum of the SHAP values equals the model output, so every prediction is split fully into feature contributions.

**LIME**

> Changes the input, watches how the prediction moves and fits a local linear model [12]. It works with any model but is approximate and random.

## 2.2 Related work

GLTR [7] showed how often each token falls in a language model's top-k predictions, and our top-1 and top-10 features come from this idea. DetectGPT [8] and Binoculars [9] detect AI text zero-shot, one from probability curvature and the other from the ratio of two models' perplexities. HC3 [1] introduced paired human and ChatGPT answers. MAGE [2] is a 27-generator bench-

mark for detection in the wild, M4GT-Bench [4] gives multi-generator and multi-domain data and HAP-E [3] gives parallel human and LLM continuations. RAID [5] showed that most detectors break under simple attacks such as paraphrasing.

Few of these works explain individual decisions. GlassQuill puts a standard supervised setup together with explanations, robustness tests and a strict focus on the false positive rate.

# 3. Dataset

## 3.1 Sources

| Corpus | Source (Hugging Face) | Licence | AI writers | Domains | Split group |
|---|---|---|---|---|---|
| HC3 | `Hello-SimpleAI/HC3` | CC BY-SA 4.0 | ChatGPT (2022) | Reddit ELI5, finance, medicine, open QA, CS Wikipedia | question |
| HAP-E | `browndw/human-ai-parallel-corpus` | MIT | GPT-4o, GPT-4o-mini, Llama-3 8B/70B | academic, news, fiction, spoken, blogs, TV/movie scripts | document |
| M4GT | `Jinyan1/COLING_2025_MGT_en (M4GT part)` | per source | GPT-4o, GPT-4, GPT-3.5, Mixtral, Llama-3, Gemma, Cohere | arXiv, essays, peer reviews, Reddit, WikiHow, Wikipedia | text |
| MAGE (test only) | `yaful/MAGE (test split)` | Apache 2.0 | 27 models incl. OPT, BLOOM, GPT-J, Flan-T5, LLaMA, GPT-3/3.5/4 | 14 domains not used in training | |

**Table 5:** Dataset sources

The HC3 and MAGE parts of the COLING data are left out so that MAGE stays a clean out-of-domain test. The pipeline downloads all corpora automatically.

## 3.2 Attributes, target variable and data types

| Attribute | Data type | Role | Description |
|---|---|---|---|
| `text` | string | input | the passage to classify |
| `label` | integer (0/1) | target | 0 = human-written, 1 = AI-generated |
| `qid` | string | grouping | question or source-document id, used only to split the data |
| `source` | categorical (17) | analysis | domain, e.g. hc3 finance, hape/news, m4gt/arxiv |
| `generator` | categorical | analysis | 'human' or the name of the model that wrote the text |
| `corpus` | categorical (3) | analysis | HC3, HAP-E or M4GT |
| `n_words` | integer | filter | number of words after cleaning (50 to 400) |

**Table 6:** Attributes, target variable and data types

The models get either the raw text (TF-IDF + LR, DistilBERT) or 18 numeric features built from it (XGBoost, section 3.6). The group id is never a model input.

## 3.3 Size, balance and exploratory analysis

| Split | Texts | HC3 | HAP-E | M4GT | Share AI |
|---|---|---|---|---|---|
| Train (70%) | 22,371 | 8,412 | 5,565 | 8,394 | 0.500 |
| Validation (15%) | 4,784 | 1,776 | 1,209 | 1,799 | 0.500 |
| Test (15%) | 4,829 | 1,808 | 1,222 | 1,799 | 0.501 |
| **Total** | **31,984** | **11,996** | **7,996** | **11,992** | **0.500** |
| MAGE (out-of-domain) | 2,000 | – | – | – | 0.500 |

**Table 7:** Dataset size per split and corpus

Every one of the 17 domains has exactly as many human texts as AI texts (Appendix B). Mean length is 256 words for human texts and 266 words for AI texts, so length alone is not a shortcut (Figure 1). The peak at 400 words is there because long texts are cut at the last full sentence.

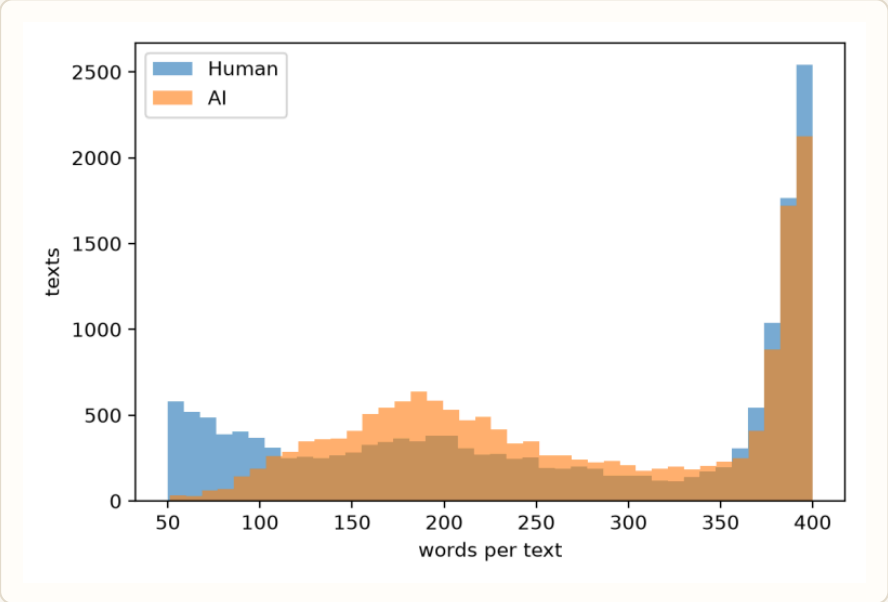



**Figure 1:** Distribution of text length (words) for human and AI texts

## 3.4 Missing values and duplicates

- Rows with empty or missing text are dropped (`dropna`) and exact duplicate texts are removed.
- After cleaning there are **0 missing values** in any split. Every attribute is fully filled in.
- The engineered features cannot be missing, because every rate divides by `max(words, 1)` and every text is at least 50 words long.

## 3.5 Preprocessing and shortcut removal

The cleaning function (`clean_text` in `src/data.py`) removes formatting that gives away where a text came from rather than how it is written. Each artifact below showed up mostly in one class and would let a model cheat. Together they explained why the first full run scored 99.3%.

| Artifact | Human texts | AI texts | Fix |
|---|---|---|---|
| literal "\n" line breaks (HC3 ChatGPT) | 0% | most | replace with a space |
| "do n't", "it 's" (tokenizer split) | 34% | 0.02% | rejoin |
| "( word )" spaced brackets | 35% | 0.01% | remove the spaces |
| URL_0 link placeholders | 11.6% | 0.07% | remove |
| markdown bold and # headings (M4GT) | 2% | 9% | remove |
| curly quotes, em dashes, [1] citations | more often | less often | normalise to plain ASCII |
| "As an AI language model...", "I hope this helps" | 0% | some | remove |

**Table 8:** Shortcut artifacts removed during preprocessing

We also remove texts outside 50–400 words. HAP-E and M4GT texts longer than 400 words are cut at the last full sentence, so long texts of both classes are treated the same way. Each domain is then **balanced** to equal human and AI counts. Without this step HC3 open_qa had 20 human texts against 764 AI texts, and the topic alone predicted the label. Last, MAGE's label convention (1 = human) is flipped to ours (1 = AI).

## 3.6 Feature engineering

XGBoost uses 18 hand-made features. Six come from GPT-2 and measure how predictable each token is. The other twelve describe writing style. Table 9 lists them with their class means on the training set and the effect size (the difference of class means in pooled standard deviations).

| Group | Feature | Meaning | Human | AI | Effect |
|---|---|---|---|---|---|
| GPT-2 | log_ppl | log perplexity (mean surprise) | 3.57 | 2.61 | −1.39 |
| | top1_frac | share of tokens that were GPT-2's first guess | 0.34 | 0.46 | +1.36 |
| | top10_frac | share of tokens in GPT-2's top 10 | 0.67 | 0.79 | +1.35 |
| | nll_std | unevenness of surprise | 2.90 | 2.37 | −1.25 |
| | mean_entropy | GPT-2's uncertainty | 3.71 | 3.27 | −1.00 |
| | nll_max | most surprising token | 13.8 | 11.7 | −0.92 |
| Rhythm | burstiness | std / mean sentence length | 0.53 | 0.38 | −0.72 |
| | std_sent_len | sentence length spread | 11.0 | 8.1 | −0.40 |
| | avg_sent_len | mean sentence length | 21.6 | 22.5 | +0.07 |
| Vocabulary | type_token_ratio | unique / total words | 0.59 | 0.52 | −0.62 |
| | function_word_rate | the, of, and, ... | 0.28 | 0.30 | +0.50 |
| | avg_word_len | letters per word | 4.64 | 4.87 | +0.42 |
| Punctuation | punct_rate | punctuation per word | 0.19 | 0.16 | −0.37 |
| | comma_rate | commas per word | 0.050 | 0.058 | +0.27 |
| Voice | contraction_rate | don't, it's, ... | 0.018 | 0.014 | −0.19 |
| | first_person_rate | I, we, my, ... | 0.016 | 0.012 | −0.16 |
| Readability | flesch_reading_ease | Flesch reading ease | 56.6 | 49.0 | −0.33 |
| | fk_grade | Flesch-Kincaid grade | 10.3 | 11.5 | +0.25 |

**Table 9:** The 18 engineered features, class means and effect sizes (training set)

## 3.7 Feature selection

We keep all 18 features. Each one was picked in advance with a stated hypothesis about how AI text differs. Tree ensembles handle correlated features well, and SHAP (section 6.6) shows afterwards which features the model really relies on. For ten features where theory and the training data agree on the direction (for example, lower perplexity and lower burstiness point to AI), XGBoost is given monotone constraints. This acts as a soft form of feature control, because these features can only push the prediction in the expected direction.

## 3.8 Validation strategy

- **Group split 70/15/15** (GroupShuffleSplit) inside each corpus. All answers to one HC3 question, or all versions of one HAP-E document, end up in the same split. The code asserts that there is no overlap.

- **5-fold GroupKFold** on the training set only, for hyper-parameter tuning, so tuning does not leak either.
- **Validation set** for the perplexity threshold, the DistilBERT epoch and the choice of the deployed verdict rule.
- **Test set** used once, at the end. MAGE and the paraphrase set measure robustness.
- **Fixed seed 42** for Python, NumPy, PyTorch, the splits and all sampling.

# 4. Methodology and ML Pipeline

## 4.1 Pipeline overview

Figure 2 shows the full pipeline. Each numbered step is a function in `run_pipeline.py`. Every step saves its output, so later steps can be rerun on their own.

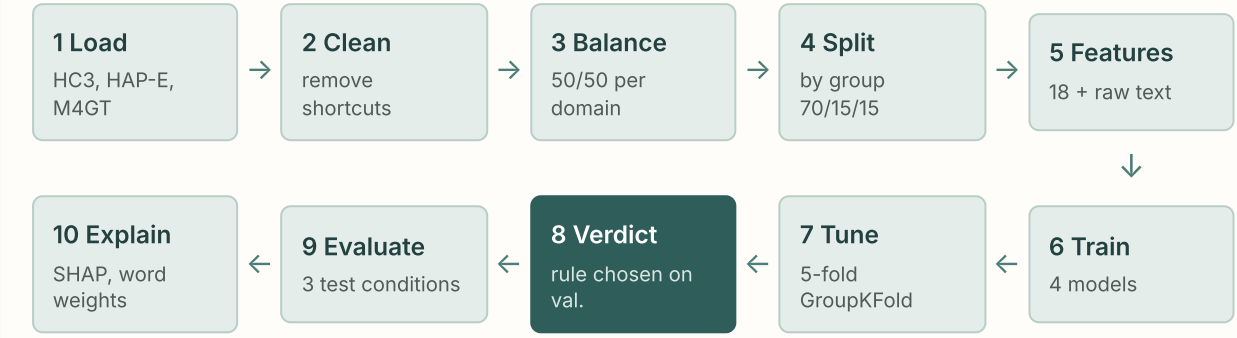



**Figure 2:** The GlassQuill ML pipeline (read the top row left to right, then the bottom row right to left)

## 4.2 Model selection and justification

The four models were picked on purpose to go from simple and transparent to powerful and opaque. This lets us measure what explainability costs (RQ2) instead of assuming it.

| # | Model | Input | Why it is included | Explained with |
|---|---|---|---|---|
| 1 | Perplexity threshold | `log_ppl` | zero-shot baseline: how far a single well-known signal goes | the value itself |
| 2 | TF-IDF (1–2-grams, 50k) + logistic regression | words | classic glass-box text model, strong on wording | exact weights (coefficient × TF-IDF) |
| 3 | XGBoost (400 trees) | 18 features | strong on tabular data and every feature has a meaning | TreeSHAP (exact) |
| 4 | DistilBERT (fine-tuned) | raw text, 256 tokens | strong black box to compare against | LIME, SHAP text |
| **5** | **Deployed verdict = mean of 2 and 3** | **both** | **the two glass boxes make different mistakes, so averaging cancels errors** | **both explanations** |

**Table 10:** Models and the reason each one is included

## 4.3 Training strategy and hyper-parameters

| Model | Training and tuning | Selected |
|---|---|---|
| Perplexity threshold | cut-off on validation by Youden's J (maximise TPR − FPR) | AI if log_ppl ≤ 2.98 |
| TF-IDF + LR | C ∈ {0.1, 1, 10}, 5-fold GroupKFold, scored on F1 | C = 10 |
| XGBoost | max_depth ∈ {3, 4, 6}, learning rate ∈ {0.05, 0.1}, 400 trees, subsample and colsample 0.8, monotone constraints, same CV | best grid point by F1 |
| DistilBERT | learning rate 2e-5, batch 16, 2 epochs, max 256 tokens, weight decay 0.01 | epoch with best validation F1 (0.964) |
| Verdict rule | compared XGBoost, LR and their mean on the validation set | mean: accuracy 0.972, FPR 0.021 |

**Table 11:** Training strategy and selected hyper-parameters

On the validation set the mean of XGBoost and LR reached accuracy 0.972 at FPR 0.021. XGBoost alone got 0.929 / 0.064 and LR alone got 0.954 / 0.032. So the mean was chosen as the deployed rule before the test set was touched. The app reports AI for p ≥ 0.65, human for p ≤ 0.35 and 'unclear' in between, so weak evidence is shown as weak instead of being forced into a class.

## 4.4 Evaluation method

Every model is scored by the same function on the same texts. The metrics are accuracy, precision, recall, F1-score, ROC-AUC, confusion matrices, ROC curves and the false positive rate (FPR). We report FPR next to every other metric because a false positive accuses a real person, and accuracy alone can hide very different error patterns.

There are three test conditions: (a) the in-domain test set overall and per corpus, (b) MAGE per generator and per domain and (c) AI texts before and after every sentence is paraphrased with a T5 paraphraser ( `humarin/chatgpt_paraphraser_on_T5_base` ).

## 4.5 Explanation method

- **Global:** TreeSHAP over the whole test set (beeswarm and bar plots) and the direction in which each feature pushes the prediction.
- **Local:** waterfall plots for one correct AI text, one correct human text, one false positive and one false negative.
- **Words:** exact per-word contributions of the logistic regression, shown as coloured text in the app.
- **Agreement checks (RQ4):** SHAP vs LIME on DistilBERT, and LIME vs the exactly known LR weights.

# 5. Implementation

## 5.1 Tools and environment

The study uses Python 3.11, PyTorch 2.11 (CUDA 12.8), Hugging Face Transformers 5.20, scikit-learn 1.9, XGBoost 3.2, SHAP 0.51, LIME 0.2, textstat and pandas. FastAPI runs the API. The website is built with React 19, TypeScript, Tailwind CSS and Motion. Quality checks use pytest, ESLint and the TypeScript compiler. Everything runs on a laptop with an NVIDIA RTX 3050 (4 GB).

## 5.2 Project structure

| Path | Responsibility |
|---|---|
| `src/data.py` | dataset URLs, loaders, clean_text, balancing (prepare), group_split |
| `src/features.py` | 12 stylometric features and the GPT-2 PerplexityScorer (6 features) |
| `src/models.py` | metrics (evaluate), perplexity threshold, TF-IDF + LR and XGBoost training with GroupKFold |
| `src/train_distilbert.py` | DistilBERT fine-tuning and batched prediction |
| `src/explain.py` | SHAP plots, exact word weights, highlights, LIME |
| `src/robustness.py` | T5 paraphraser for the paraphrase attack |
| `src/detector.py` | loads the saved models once and analyses one text for the app |
| `run_pipeline.py` | the whole study: 8 stages, cached outputs, `--quick` mode, fixed seed |
| `server.py` | FastAPI app: input validation, rate limiting, security headers, serves the website |
| `web/` | React website: home, detector, results and method pages |
| `tests/test_server.py` | 28 automated API tests |
| `models/, reports/` | trained models, result tables, figures and run logs |

**Table 12:** Project structure

## 5.3 Workflow and reproducibility

The full study runs with one command. Each stage writes its outputs to disk and is skipped or reused on the next run, so any stage can be repeated by itself:

```
python run_pipeline.py --quick      # all 8 stages on a small sample, about 10 minutes
python run_pipeline.py              # the full study, about 1.5 hours on the laptop GPU
python run_pipeline.py --stages evaluate ood paraphrase explain
python -m pytest tests -q           # 28 API tests
python server.py                    # web app at http://127.0.0.1:8000
```

The stages are data → features → train → bert → evaluate → ood → paraphrase → explain. Datasets download automatically, GPT-2 features are cached per split and all result tables are collected into `reports/RESULTS.md`. The trained models are committed, so the web app runs without retraining.

## 5.4 The web application

A user pastes a text. The server removes invisible characters and checks that the input is English prose of at least 50 words. It then computes the 18 features, runs XGBoost and the word model and returns the verdict. The verdict comes with an 'unclear' band, both model scores, the top reasons compared with typical human and AI values, a SHAP chart and word-level evidence (Figures 3 and 4). Each analysis takes about 0.1 s on the GPU. Pasted text is never stored or logged.

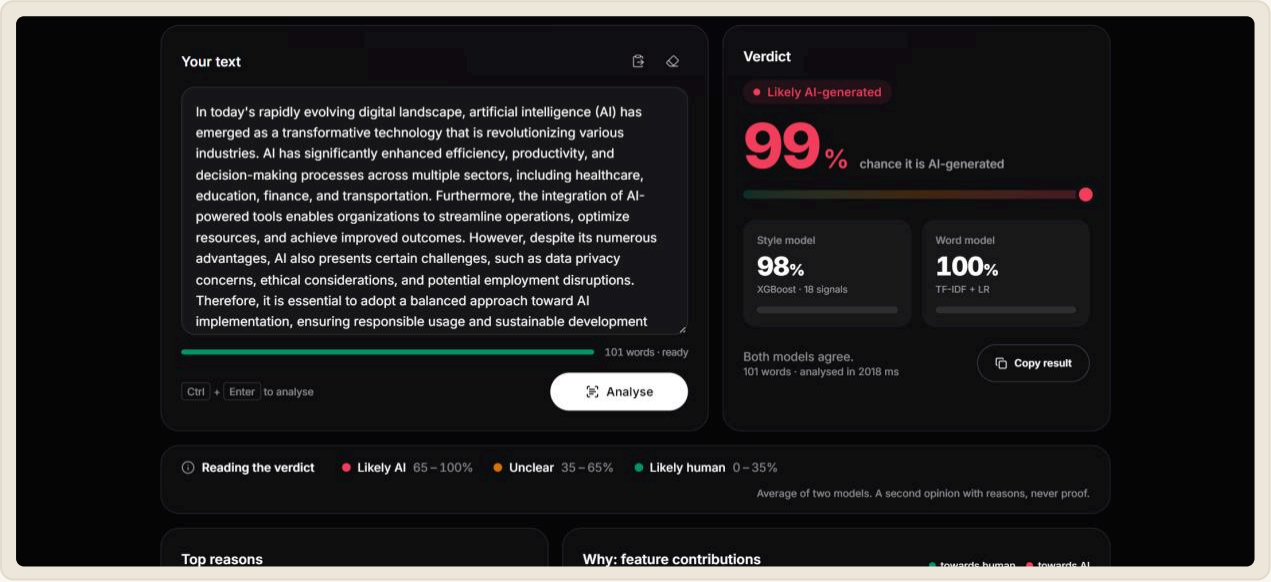



**Figure 3:** Detector page: verdict, unclear band and the scores of both models

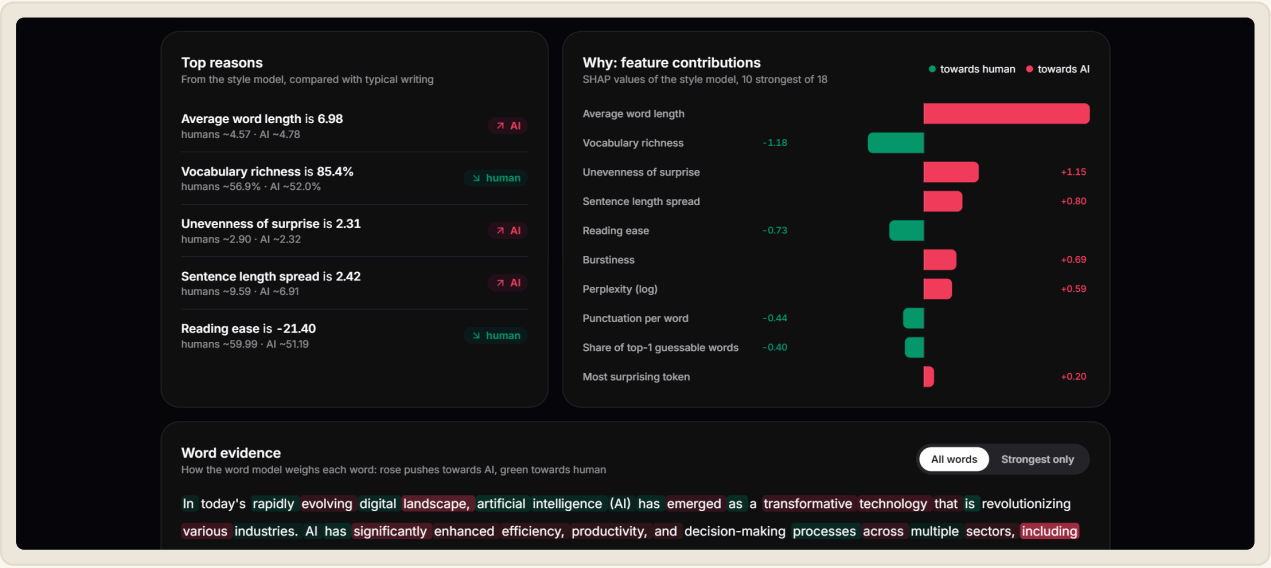



**Figure 4:** Explanations in the app: top reasons and SHAP feature contributions

## 5.5 Testing and security

- 28 automated tests cover input validation (too short, too long, wrong type, not English), behaviour (a known AI text is flagged and a human text is not, and invisible characters do not change the verdict) and security.
- The security measures are a request size cap, a mandatory Content-Length, a per-IP rate limit (20 analyses per minute), a strict Content-Security-Policy with other security headers, path-traversal protection and error messages that never echo the submitted text.

# 6. Results and Comparative Analysis

## 6.1 In-domain test set

| Model | Accuracy | Precision | Recall | F1 | ROC-AUC | FPR |
|---|---|---|---|---|---|---|
| Perplexity threshold | 0.839 | 0.892 | 0.771 | 0.827 | 0.902 | 0.093 |
| TF-IDF + LR | 0.959 | 0.970 | 0.948 | 0.959 | 0.990 | 0.030 |
| XGBoost | 0.923 | 0.933 | 0.912 | 0.923 | 0.978 | 0.066 |
| DistilBERT | 0.961 | 0.934 | 0.993 | 0.963 | 0.997 | 0.071 |
| **GlassQuill verdict (XGBoost + LR)** | **0.974** | **0.980** | **0.968** | **0.974** | **0.995** | **0.020** |

**Table 13:** Results on the in-domain test set (4,829 texts)

The deployed verdict has the best accuracy, precision and F1 and the lowest false positive rate. DistilBERT has the highest recall and ROC-AUC, but it wrongly flags 7.1% of human texts, which is three and a half times as many as the verdict. Averaging the two explainable models does better than either one on its own because XGBoost and the word model make different mistakes.

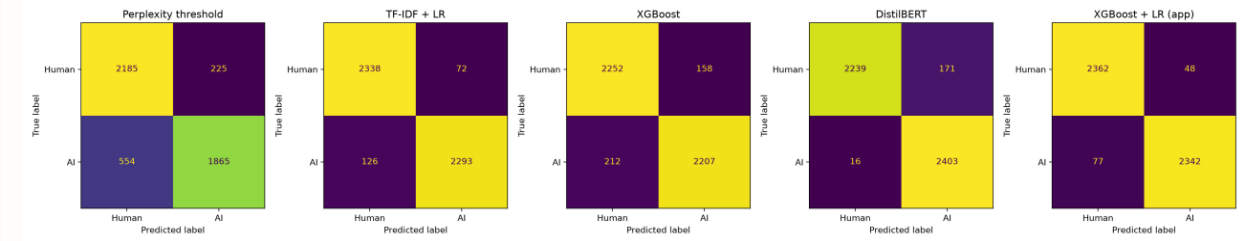



**Figure 5:** Confusion matrices on the test set (rows: true class, columns: predicted class)

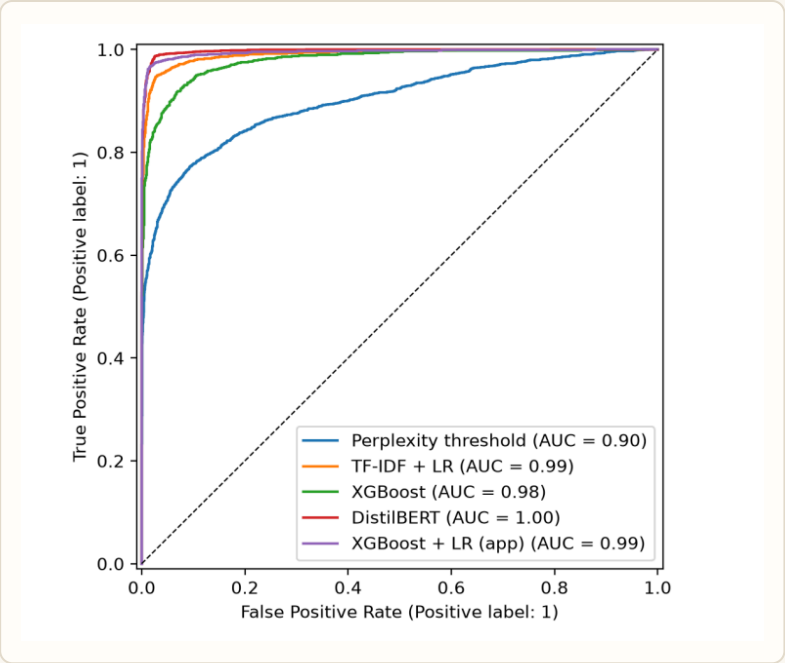



**Figure 6:** ROC curves on the test set

## 6.2 Results per corpus

| Model | HC3 | HAP-E | M4GT |
|---|---|---|---|
| Perplexity threshold | 0.958 | 0.606 | 0.800 |
| TF-IDF + LR | 0.973 | 0.977 | 0.932 |
| XGBoost | 0.964 | 0.893 | 0.898 |
| DistilBERT | 0.970 | 0.973 | 0.949 |
| **GlassQuill verdict** | **0.987** | **0.981** | **0.956** |

**Table 14:** F1 score per corpus on the test set

The perplexity threshold works well on 2022 ChatGPT text (HC3) but drops to 0.61 F1 on HAP-E, where LLMs continue a human text in the same style. The verdict is the best model on every corpus.

## 6.3 Out-of-domain: MAGE

| Model | Accuracy | Precision | Recall | F1 | ROC-AUC | FPR |
|---|---|---|---|---|---|---|
| Perplexity threshold | 0.656 | 0.792 | 0.422 | 0.551 | 0.681 | 0.111 |
| TF-IDF + LR | 0.606 | 0.787 | 0.292 | 0.426 | 0.648 | 0.079 |
| XGBoost | 0.634 | 0.736 | 0.416 | 0.532 | 0.611 | 0.149 |
| DistilBERT | 0.655 | 0.734 | 0.486 | 0.585 | 0.696 | 0.176 |
| **GlassQuill verdict** | **0.664** | **0.854** | **0.397** | **0.542** | **0.655** | **0.068** |

**Table 15:** Results on MAGE: 2,000 texts from 27 generators and 14 domains never used in training

Every model falls to 61–66% accuracy on unseen generators, which confirms that AI-text detection does not transfer easily. The verdict still has the best accuracy and precision and by far the lowest false positive rate (6.8%), while DistilBERT's FPR rises to 17.6%.

Accuracy per generator family shows where the errors come from: GPT-4 0.91, GLM-130B 0.81, GPT-3.5 0.80, GPT-3 davinci 0.76, LLaMA 0.54, GPT-4 paraphrase 0.49, T0 0.47, Flan-T5 0.35, OPT 0.04, BLOOM 0.02, GPT-J/NeoX 0.00 and human texts 0.93. Instruction-tuned chat models, which are the ones students actually use, get caught. Small base models that sample unusual words look human to every detector. Compared with the HC3-only Phase I model, GPT-4 detection rose from 0.55 to 0.91 and GPT-3.5 detection rose from 0.55 to 0.80.

## 6.4 Paraphrase attack

| Model | Recall before | Recall after | Drop |
|---|---|---|---|
| Perplexity threshold | 1.00 | 0.93 | −0.07 |
| TF-IDF + LR | 0.93 | 0.77 | −0.17 |
| XGBoost | 1.00 | 0.95 | −0.05 |
| DistilBERT | 1.00 | 1.00 | 0.00 |
| **GlassQuill verdict** | **0.98** | **0.93** | **−0.05** |

**Table 16:** Recall on 60 AI test texts before and after paraphrasing every sentence

Paraphrasing hurts the word model the most (−17 points) because the exact words change. The perplexity and style features mostly survive (−5 to −7 points). DistilBERT is not affected on this small set. The verdict still catches 93% of paraphrased AI texts.

## 6.5 Comparative analysis

- **Explainable vs black box (RQ2):** XGBoost alone trails DistilBERT (0.923 vs 0.963 F1). The explainable verdict beats it (0.974 F1) with less than a third of its false positives, and it does so

on every corpus.

- **Precision–recall trade-off:** DistilBERT leans towards recall (0.993) and pays for it in precision (0.934). The verdict is balanced (0.980 / 0.968), which is the right profile when a false accusation is the costly error.
- **Robustness:** the in-domain gaps shrink out-of-domain, where all models land between 61% and 66%. There the verdict's edge is its low FPR. Paraphrasing separates the models by what they read: words are fragile while style and perplexity hold up.
- **Data quality vs model choice:** removing shortcuts changed the results more than any change of model. The uncleaned data gave a misleading 99.3%, and adding modern corpora fixed the failure on current ChatGPT output.

## 6.6 Explanations

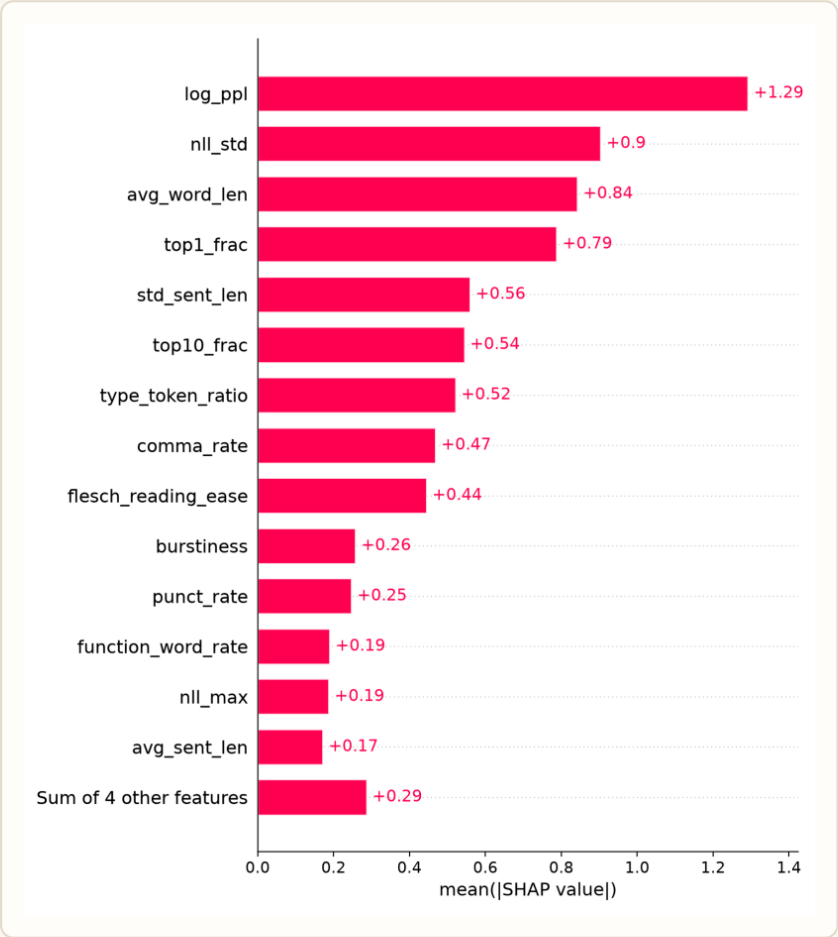



**Figure 7:** Global feature importance of XGBoost (mean |SHAP| over the test set)

- **Global SHAP:** the most important features are log perplexity (low → AI), nll_std (even surprise → AI), average word length (long words → AI), top1_frac (guessable words → AI) and sentence-length spread (even → AI).
- **Glass-box words:** the strongest AI words in the logistic regression include 'including', 'important to', 'overall' and 'significant'. Human words include 'etc', 'basically', 'since' and 'thus'.

- **SHAP vs LIME on DistilBERT:** the top-10 word overlap (Jaccard) is only 0.06. Part of the reason is that DistilBERT works on word pieces while LIME removes whole words.
- **LIME vs exact LR weights:** LIME recovers 51% of the true top-10 words, so the app uses exact SHAP values and exact word weights instead.

## 6.7 Answers to the research questions

| RQ | Answer |
|---|---|
| RQ1 | GPT-2 surprise features separate the classes best (about 1.3 SD: log perplexity, top-1/top-10 share, nll_std). Burstiness and vocabulary richness help a moderate amount, but sentence length alone does not. H1 and H2 are confirmed by both the class means and the SHAP direction. |
| RQ2 | An explainable system can match the black box. The verdict reaches 0.974 F1 at 2.0% FPR against DistilBERT's 0.963 F1 at 7.1% FPR. |
| RQ3 | All models drop to 61–66% accuracy on unseen generators, but chat models are still caught (GPT-4 91%). Paraphrasing lowers recall for every model except DistilBERT, so H4 mostly holds. H3 is partly confirmed: DistilBERT is not the best in-domain on accuracy, but it has the highest FPR on MAGE (17.6%). |
| RQ4 | No. SHAP and LIME barely agree on DistilBERT (Jaccard 0.06), and LIME recovers only half of the known LR words. |

**Table 17:** Answers to the research questions

# 7. Limitations and Ethical Considerations

- A detector score must never be the only evidence against a person. The app says this on every page and shows an 'unclear' band instead of guessing.
- Simple and predictable wording, which is common in non-native writing, has low perplexity and can lead to false positives. We report FPR everywhere, but we did not test fairness on non-native essays directly.
- Small open models with sampled output (OPT, BLOOM, GPT-J) and paraphrasing tools fool all four models. New LLMs come out quickly and can make these results out of date.
- The detector works on English only and only on the genres in the training corpora. Texts that mix human and AI writing are not handled.
- HAP-E's AI texts continue a human text in the same style. This is harder than spotting a normal chatbot answer, so the results on HAP-E are a lower bound.
- Privacy: submitted text is processed in memory and is never stored or logged.

# 8. Conclusion and Future Scope

## 8.1 Conclusion

GlassQuill shows that an explainable detector can match a fine-tuned transformer. Averaging an XGBoost model on 18 meaningful features with a word-level logistic regression gives 97.4% accuracy, 0.974 F1 and a 2.0% false positive rate on 4,829 unseen texts. That puts it ahead of DistilBERT, and every decision comes with reasons at the feature level and the word level.

Two findings mattered most. First, removing formatting shortcuts from the data turned a misleading 99.3% into an honest result. Second, training on modern LLM output with direction-constrained features fixed the Phase I model's failure on current ChatGPT text. The main weak spots are unseen generator families and paraphrasing. The project is delivered as a reproducible pipeline, a tested web application and complete documentation.

## 8.2 Future scope

1. Add a stronger reader model or Binoculars (a two-model perplexity ratio) as extra features. An early test gave +0.01 to +0.04 AUC at about 25 times the cost.
2. Set the decision threshold for a fixed and very low false positive rate (for example 1%) instead of 0.5.
3. Score each sentence to find AI-written passages inside human texts.
4. Evaluate on the RAID benchmark and add newer generators to training.
5. Calibrate the probabilities so that '80%' really means 80%, and test fairness on non-native English essays.
6. Extend the detector to Indian languages.

# References


[1] B. Guo et al., "How close is ChatGPT to human experts? Comparison corpus, evaluation, and detection," arXiv:2301.07597, 2023.

[2] Y. Li et al., "MAGE: Machine-generated text detection in the wild," in Proc. ACL, 2024.

[3] A. Reinhart, D. W. Brown et al., "Do LLMs write like humans? Variation in grammatical and rhetorical styles," arXiv:2410.16107, 2024.

[4] Y. Wang et al., "M4GT-Bench: Evaluation benchmark for black-box machine-generated text detection," in Proc. ACL, 2024.

[5] L. Dugan et al., "RAID: A shared benchmark for robust evaluation of machine-generated text detectors," in Proc. ACL, 2024.

[6] W. Liang et al., "GPT detectors are biased against non-native English writers," Patterns, vol. 4, no. 7, 2023.

[7] S. Gehrmann, H. Strobelt and A. M. Rush, "GLTR: Statistical detection and visualization of generated text," in Proc. ACL System Demonstrations, 2019.

[8] E. Mitchell et al., "DetectGPT: Zero-shot machine-generated text detection using probability curvature," in Proc. ICML, 2023.

[9] A. Hans et al., "Spotting LLMs with Binoculars: Zero-shot detection of machine-generated text," in Proc. ICML, 2024.

[10] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in Proc. NeurIPS, 2017.

[11] S. M. Lundberg et al., "From local explanations to global understanding with explainable AI for trees," Nature Machine Intelligence, vol. 2, pp. 56–67, 2020.

[12] M. T. Ribeiro, S. Singh and C. Guestrin, "Why should I trust you? Explaining the predictions of any classifier," in Proc. KDD, 2016.

[13] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in Proc. KDD, 2016.

[14] V. Sanh et al., "DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter," arXiv:1910.01108, 2019.

[15] A. Radford et al., "Language models are unsupervised multitask learners," OpenAI technical report, 2019.

# Appendix A. Rubric Coverage

| ESE component (marks) | Where it is addressed |
|---|---|
| Problem refinement and use case relevance (3) | Ch. 1: problem, relevance, objective, scope, domain, stakeholders, refinement since Phase I (Table 4) |
| Dataset understanding, preprocessing, feature handling (4) | Ch. 3: sources, size, attributes, target, data types, missing values, preprocessing, feature engineering and selection, validation strategy |
| Methodology and ML pipeline design (5) | Ch. 4: pipeline figure, model selection and justification, training strategy, evaluation and explanation methods |
| Implementation / project execution (5) | Ch. 5: tools, project structure, reproducible workflow, web application, tests and security |
| Results, evaluation metrics, comparative analysis (5) | Ch. 6: accuracy, precision, recall, F1, ROC-AUC, FPR, confusion matrices, ROC curves, per-corpus, out-of-domain, paraphrase, explanations, RQ answers |
| Final report quality and documentation (3) | this report, plus README, code walkthrough and handoff guide in the repository |
| Communication and presentation (2) | slide deck (submissions/) with agenda, timing and live demo |
| Viva / project defence (3) | every design choice is justified in Ch. 3–4, with limitations and future scope in Ch. 7–8 |

**Table 18:** Where each ESE rubric component is addressed

# Appendix B. Class Balance per Domain

| Domain | Human | AI |
|---|---|---|
| hc3: reddit_eli5 | 4,623 | 4,623 |
| hc3: finance | 878 | 878 |
| hc3: medicine | 273 | 273 |
| hc3: wiki_csai | 191 | 191 |
| hc3: open_qa | 33 | 33 |
| hape/acad | 592 | 592 |
| hape/blog | 736 | 736 |
| hape/fic | 673 | 673 |
| hape/news | 637 | 637 |
| hape/spok | 830 | 830 |
| hape/tvm | 530 | 530 |
| m4gt/arxiv | 1,553 | 1,553 |
| m4gt/reddit | 1,389 | 1,389 |
| m4gt/wikihow | 1,385 | 1,385 |
| m4gt/wikipedia | 1,327 | 1,327 |
| m4gt/peerread | 196 | 196 |
| m4gt/outfox | 146 | 146 |
| **Total (17 domains)** | **15,992** | **15,992** |

**Table 19:** Number of human and AI texts per domain after cleaning and balancing